

API Security Risk Analysis Report

Read-Only API Security Assessment — JSONPlaceholder

Cyber Security Task 3 — Future Interns

Prepared by: Kabi

Program: Cyber Security Internship

Date: October 2026

Target: <https://jsonplaceholder.typicode.com>

1. Executive Summary

This report presents a read-only API security risk analysis of JSONPlaceholder, a public demo REST API, performed the way a security consultant would assess a client's SaaS API. The goal was to identify common, realistic API risks — unauthenticated access, data exposure, broken authorization, weak input validation, and missing protective controls — and translate them into business impact and remediation guidance.

Seven findings were identified, ranging from High to Low severity, plus one control that was tested and found to be working correctly. The most significant issues are a complete lack of authentication on endpoints returning full personal data, and the ability to access any user's resources simply by changing a numeric ID in the URL — a classic Broken Object Level Authorization (BOLA) pattern, the #1 risk on the OWASP API Security Top 10.

2. Objective & Scope

Objective: analyze a public API, identify common API security risks, assess authentication and access control, explain risks in business language, and suggest remediation — thinking like a security consultant, not attempting to breach anything.

Allowed

- Testing public or demo APIs only
- Read-only requests (GET), and safe POST requests where the API explicitly allows them without persisting real data
- Documentation-based analysis
- Header, token, and response inspection

Not Allowed (and not performed)

- Exploitation or bypass attempts
- Flooding / DoS testing
- Attacking private or production APIs

All testing in this report was performed against JSONPlaceholder, a public API explicitly built and maintained for testing and learning. No real user data, private systems, or production services were accessed.

3. Methodology & Tools Used

- Sent requests using Postman and inspected status codes, response bodies, and response headers for each test.
- Tested authentication requirements by calling endpoints with no credentials or tokens supplied.
- Tested authorization by requesting the same resource type under different numeric IDs (e.g. /users/1/posts vs /users/2/posts) with no session or ownership context.

- Tested input handling by submitting a write request (POST) with an unexpected data type and an unescaped script tag.
- Tested error handling by requesting out-of-range and non-numeric resource IDs.
- Tested for rate limiting by sending repeated requests and inspecting rate-limit headers.
- Inspected response headers for the presence or absence of standard security headers and for information disclosure.

4. API Tested

Target: <https://jsonplaceholder.typicode.com>

Type: Public fake REST API (json-server), intended for testing and prototyping — not a real production service

Endpoints assessed: /users, /users/{id}/posts, /posts, /posts/{id}/comments

Authentication in place: None — no API key, token, or login mechanism exists on any tested endpoint

5. Risk Classification Framework

- High — directly exposes sensitive data or allows unauthorized access to other users' resources with no mitigating control.
- Medium — a real weakness that enables further attacks (e.g. stored XSS, scraping) but requires an additional step or specific context to cause harm.
- Low — a hardening gap or informational disclosure that offers attackers a head start but is not independently exploitable.

6. Findings

6.1 Unauthenticated Access to Full User PII

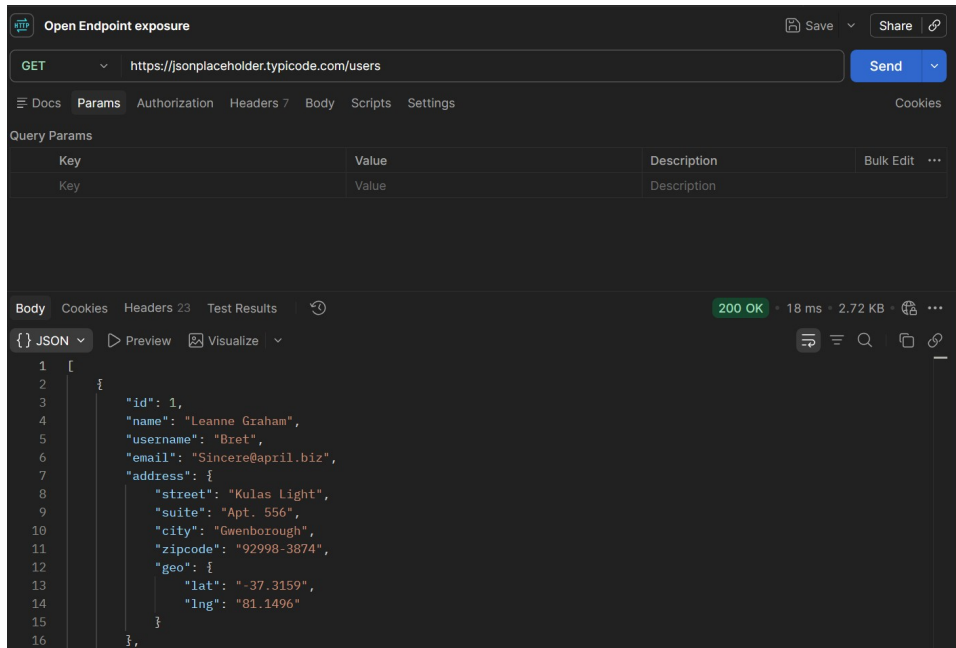


Figure 1: GET /users returns full personal data for all 10 users with no credentials supplied.

Risk identified: The /users endpoint returns full name, email, phone number, home address, and precise GPS coordinates (latitude/longitude) for every user, to any caller, with zero authentication

Severity: **High**

Business impact: If this pattern existed in a real product, anyone could scrape the entire user base's contact and location data without logging in — enabling spam, targeted phishing (directly relevant to the Task 2 analysis), doxxing, or resale of harvested data. This is also a regulatory exposure (GDPR/CCPA-style) in most jurisdictions.

Remediation: Require authentication (API key or OAuth token) on every endpoint returning personal data. Apply field-level filtering so responses return only the minimum data needed for the calling use case, and scope access by permission level.

6.2 Excessive Data Exposure in Nested Comments Endpoint

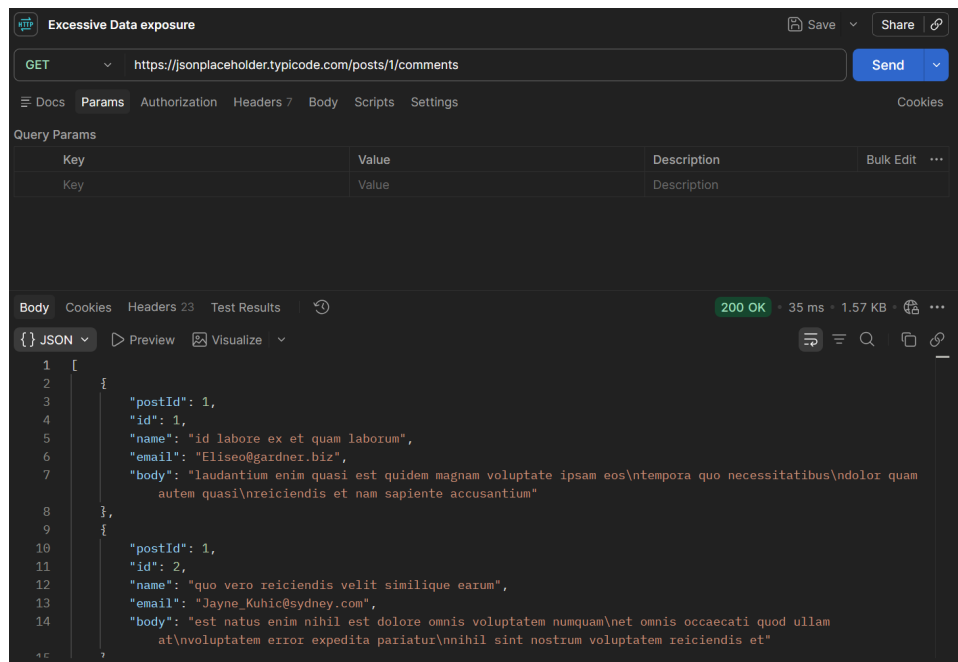


Figure 2: GET /posts/1/comments returns commenter email addresses with no authentication.

Risk identified: A secondary, nested endpoint independently leaks personal data (commenter email addresses), showing the exposure is not limited to one endpoint

Severity: **High**

Business impact: Attackers don't need to find the 'main' user endpoint — any nested or secondary endpoint can leak the same class of data, widening the attack surface for the email-harvesting scenario described above.

Remediation: Apply the same authentication and field-filtering controls consistently across every endpoint that touches personal data, not just the primary resource — audit nested/child resources specifically, since they are often overlooked.

6.3 Broken Object-Level Authorization (IDOR)

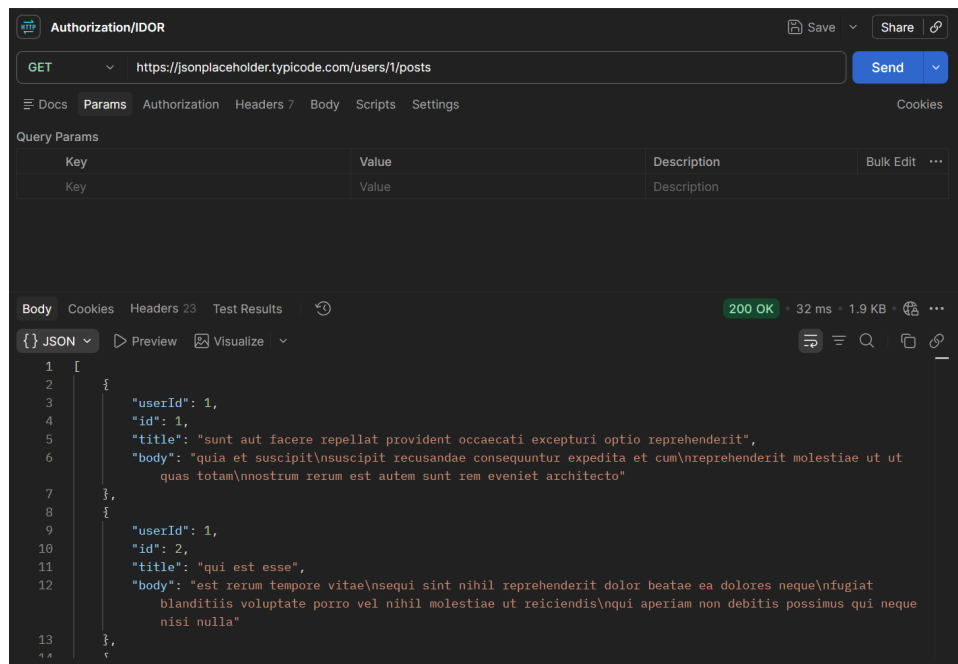


Figure 3: GET /users/1/posts returns user 1's full post history with no ownership check.

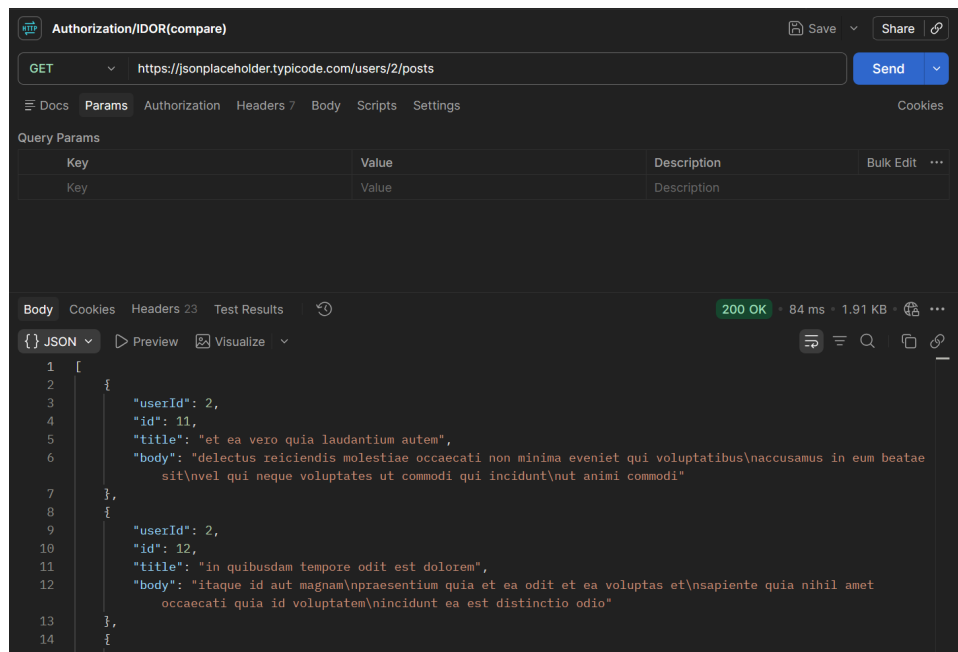


Figure 4: Simply changing the ID to /users/2/posts returns a different user's data just as freely.

Risk identified: Any resource belonging to any user can be retrieved by incrementing a numeric ID in the URL — there is no session, token, or ownership check of any kind

Severity: Medium-High

Business impact: This is OWASP API Security Risk #1: Broken Object Level Authorization (BOLA). In a real application with private data (draft posts, messages,

billing records), this exact pattern lets any authenticated — or even unauthenticated — user enumerate IDs and pull every other user's private records.

Remediation: Enforce object-level authorization on the server for every request: verify the authenticated caller is actually permitted to access the specific resource ID requested, not just that a valid session exists. Avoid predictable sequential IDs for sensitive resources where feasible.

6.4 Insufficient Input Validation on Write Endpoint

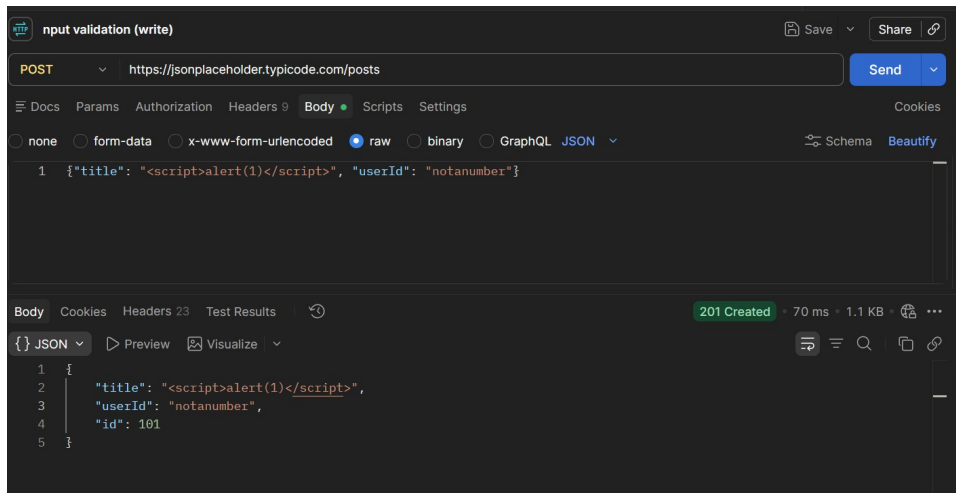


Figure 5: POST /posts with an unescaped <script> tag and a non-numeric userId was accepted (201 Created) without rejection.

Risk identified: The API accepted a raw, unescaped <script> tag in the title field and a string ("notanumber") in place of a numeric userId, returning 201 Created in both cases with no validation error

Severity: Medium

Business impact: If a real application stored and later rendered this field (e.g. in an admin panel or comment view) without separately sanitizing it, this pattern enables stored XSS. The silent acceptance of a wrong data type also signals weak schema enforcement, which can cause downstream errors or data corruption in other systems consuming this API.

Remediation: Enforce strict server-side schema validation — reject fields with unexpected types instead of silently accepting them. Sanitize or encode any user-supplied text before storage and especially before rendering it anywhere.

6.5 Backend Framework Disclosure

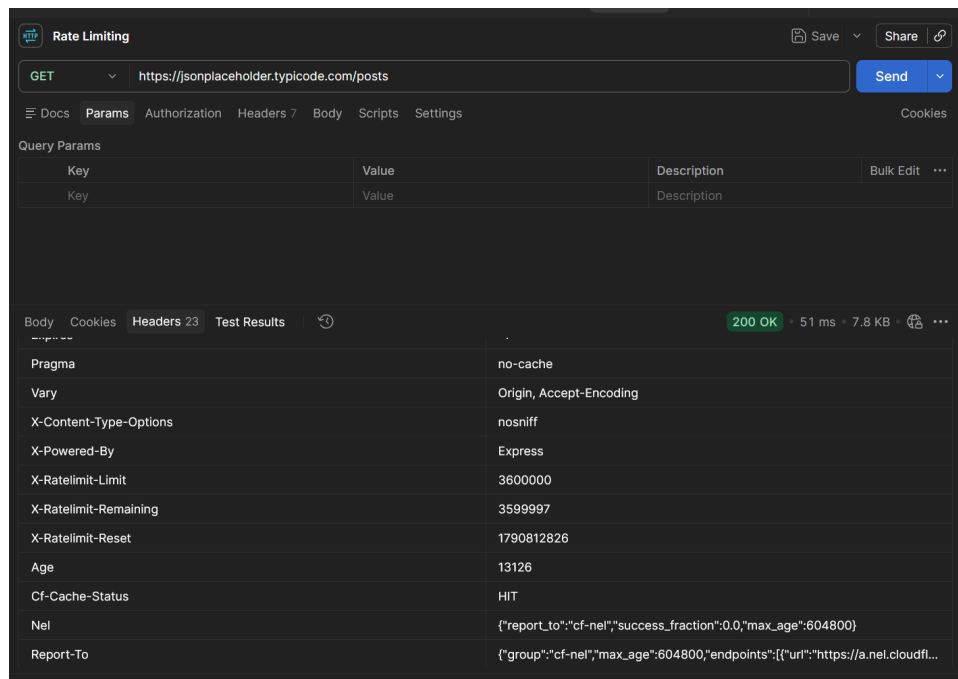


Figure 6: The X-Powered-By header discloses that the backend runs Express.

Risk identified: The X-Powered-By: Express header is present on every response, revealing the exact backend framework

Severity: Low

Business impact: On its own this is minor, but it gives an attacker a head start narrowing which framework-specific vulnerabilities or misconfigurations to try first — reconnaissance that should not be handed out for free.

Remediation: Disable identifying headers at the application layer (e.g. `app.disable('x-powered-by')` in Express) or strip them at the reverse proxy / CDN layer.

6.6 Missing Security Headers

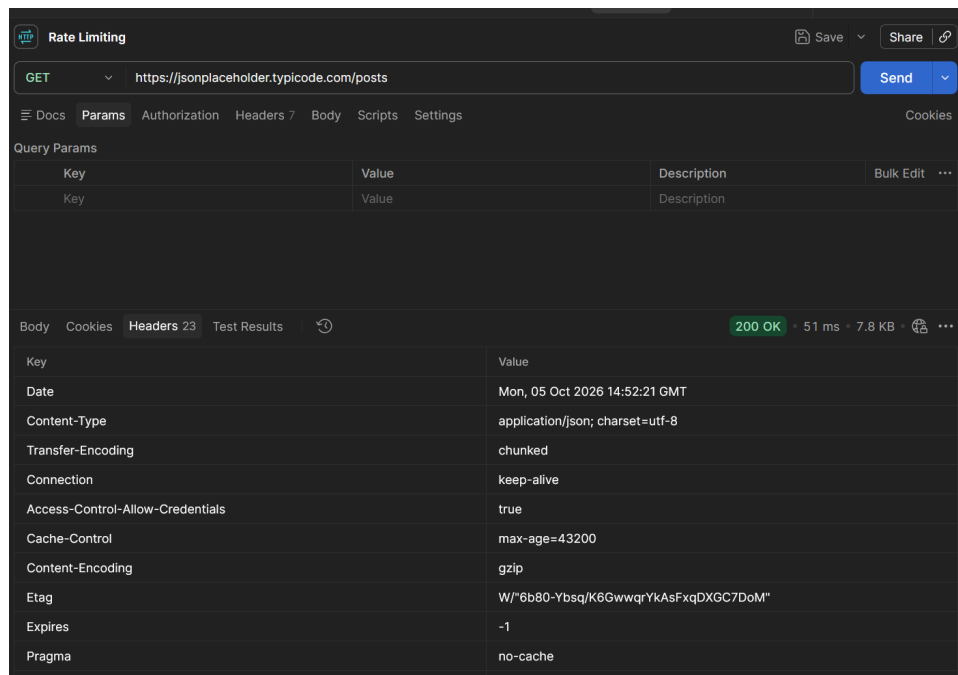


Figure 7: Full response header set — no *X-Frame-Options*, *Content-Security-Policy*, or *Strict-Transport-Security* present.

Risk identified: Across all tested responses, standard protective headers — *X-Frame-Options*, *Content-Security-Policy*, and *Strict-Transport-Security* — are absent

Severity: **Low**

Business impact: Browser-based clients consuming this API gain no protection against clickjacking (missing *X-Frame-Options*) or content-injection style attacks (missing CSP), and connections are not enforced over HTTPS at the application level (missing HSTS).

Remediation: Add standard security headers at the application or edge/CDN layer: *X-Frame-Options*: DENY (or equivalent CSP frame-ancestors directive), a defined *Content-Security-Policy*, and *Strict-Transport-Security*.

6.7 Rate Limiting Present but Set Too High to Be Effective

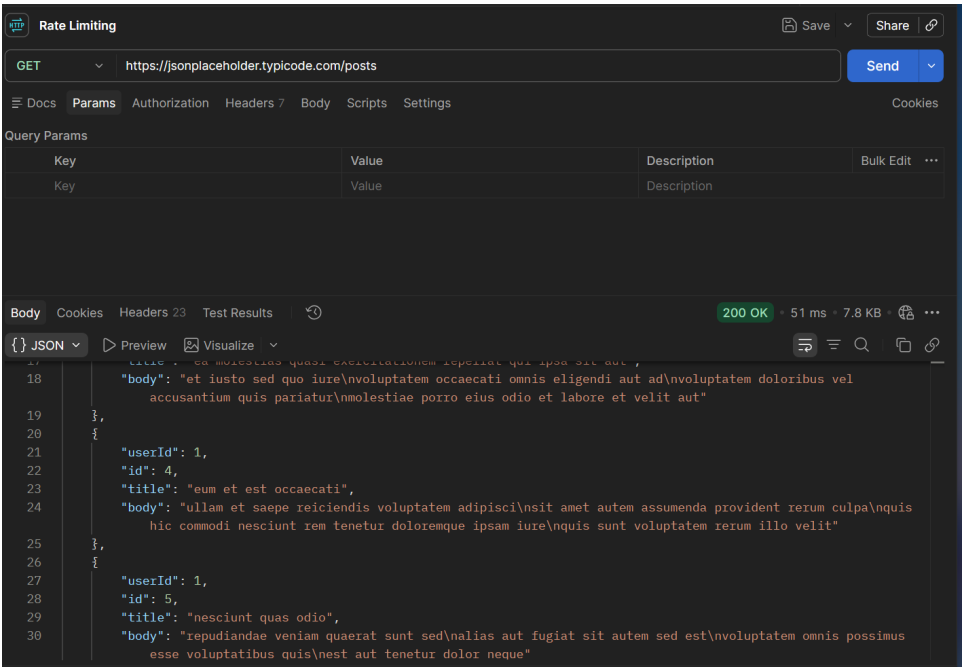


Figure 8: Repeated requests to /posts all return 200 OK with no throttling.

Risk identified: Rate-limit headers show a ceiling of X-RateLimit-Limit: 3,600,000, confirming a mechanism exists, but the threshold is high enough that it offers no meaningful protection against automated scraping

Severity: Low-Medium

Business impact: Given Findings 6.1 and 6.2 (full PII exposed with no auth), a properly-tuned rate limit is one of the few controls that could slow down automated harvesting of that data. At 3.6 million requests, that protection is effectively theoretical.

Remediation: Lower rate limits to realistic per-IP / per-API-key thresholds based on expected legitimate traffic, with tighter limits specifically on endpoints that return personal data.

6.8 Tested and Passed: Error Handling on Invalid IDs

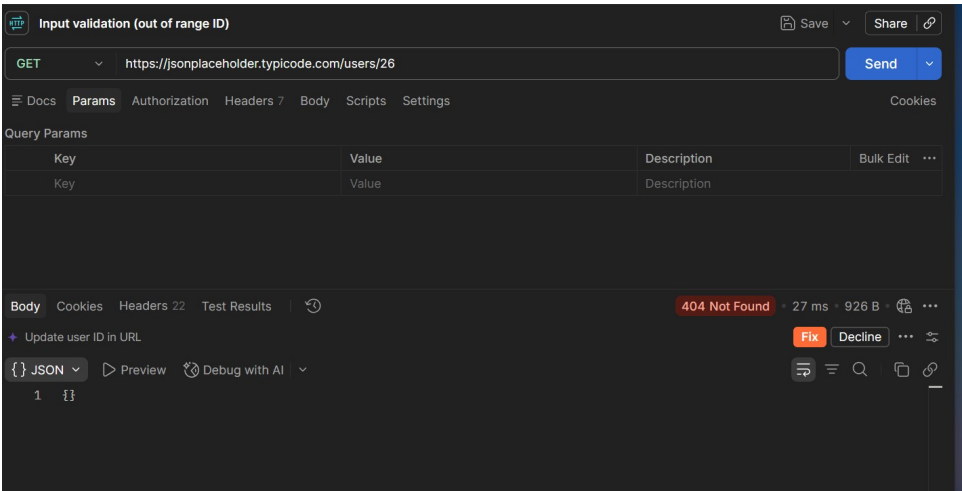


Figure 9: GET /users/26 (out-of-range ID) returns a clean 404 with no error detail leaked.

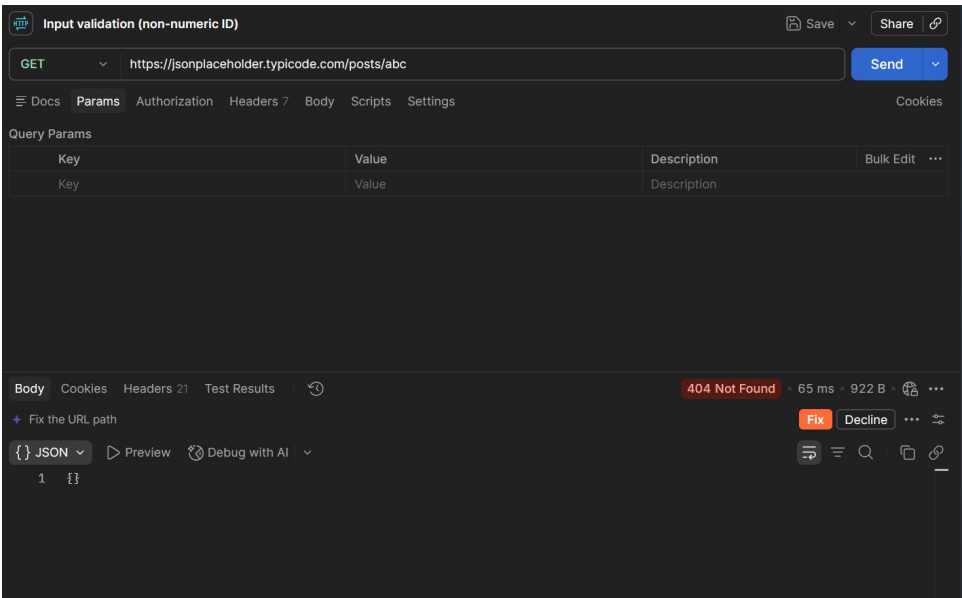


Figure 10: GET /posts/abc (non-numeric ID) also returns a clean 404.

Out-of-range and non-numeric resource IDs were tested against both /users and /posts. Both returned standard 404 Not Found responses with empty bodies — no stack trace, server path, or internal detail was disclosed. This control is working correctly and is listed here for completeness, not as a risk requiring remediation.

7. Summary of Findings

Finding	Risk Category	Severity
Unauthenticated access to full user PII (/users)	Broken Authentication / Data Exposure	High
Excessive data exposure in nested endpoint	Data Exposure	High

(/posts/{id}/comments)		
Broken Object-Level Authorization (/users/{id}/posts)	Broken Authorization (BOLA)	Medium-High
Insufficient input validation on write (POST /posts)	Input Validation	Medium
Rate limiting threshold too high to be effective	Rate Limiting	Low-Medium
Backend framework disclosure (X-Powered-By)	Information Disclosure	Low
Missing security headers (X-Frame-Options / CSP / HSTS)	Security Misconfiguration	Low

8. Remediation Priorities

- Priority 1 — Add authentication to every endpoint returning personal data, and enforce object-level authorization so one user's token cannot retrieve another user's resources by ID.
- Priority 2 — Add server-side schema validation and output sanitization on all write endpoints.
- Priority 3 — Tighten rate-limit thresholds, add missing security headers, and remove framework-identifying headers.

9. Conclusion

None of the issues found here required exploitation, bypassing, or any tool beyond a standard HTTP client — every finding was visible from a plain, authorized GET or POST request. That is the core lesson of an API security review: the most damaging issues are usually missing controls (authentication, authorization, validation), not clever attacks. Prioritizing authentication and object-level authorization first would close the two highest-severity gaps identified in this assessment.